How This Thesis Was Actually Built

A step-by-step account, in plain language

Mohamed Naceur Hammami

August 18, 2026

Abstract

This document explains, without assuming any background, what was done in this project, what went wrong, how each problem was found, and what was learned. It is written to be read on its own. Technical terms are explained the first time they appear.

It exists because the interesting part of this project is not the final number. It is the sequence of six defects that each produced believable but wrong results, and the measurements that eventually caught them. Anyone building a similar system will meet the same traps.

Contents

1 The question, and why it is hard

1.1 What we wanted to know

When you reach out to touch something, your brain has to decide how hard to pull with each of dozens of muscles. Working out those forces is a standard problem in biomechanics, and it is normally solved with mathematics that is accurate but slow. This project asked a simple question:

> *Can a neural network, trained by trial and error, work out the same muscle forces — and does its answer agree with the mathematical one?*

1.2 Why it is not obvious

Your arm has more muscles than it has ways to bend. In our model: nine muscles, four axes of rotation. That means **infinitely many combinations of muscle pulls produce exactly the same arm movement**.

> Picture nine people pulling ropes tied to one heavy door, and you want the door at a particular angle. They could all pull gently. Three could pull hard while six rest. Or — wastefully — some could pull one way while others pull the opposite way, cancelling out, everyone exhausted, the door in the same place. From across the room, all three look identical.

This is called the **muscle redundancy problem**. Because the movement alone does not tell you the forces, you need an extra rule to pick one answer. The classical rule, in use since 1981, is *minimum effort*: of all the combinations that work, choose the one where the muscles work least hard. The method built on that rule is called **static optimisation**, and it is what this project compares against.

2 Step 0: discovering the starting point was fiction

Before any new work could begin, the existing results had to be checked.

The previous version of this thesis reported a comparison of four learning algorithms, with statistical tests, across ten repeated experiments. It reported that the best method succeeded 91.4 % of the time and stopped within 1.4 cm of the target.

Reading the code that produced those numbers revealed lines of this form:

```
# Simulated multi-seed results (replace with real evaluation data)
np.clip(rng.normal(0.82, 0.05, 10), 0, 1)
```

In plain terms: **the results were random numbers drawn from a bell curve with a hand-chosen average.** The comment shows they were always meant to be replaced by real measurements. That replacement never happened.

Supporting checks confirmed it. No code existed anywhere that trained three of the four algorithms. No code existed to compute the muscle-synergy or co-contraction analyses the text described. Two trained models existed on disk, against the forty the text said were archived.

The lesson. Before building on any result, find the line of code that produced it. A number in a document is not evidence; the computation that generated it is.

3 Step 1: six defects, each of which looked fine

With the old results discarded, the system had to be made to work. Six separate faults were found. What makes them worth documenting is that *four of them produced perfectly believable results while being completely wrong.*

3.1 Defect 1 — the arm was frozen, in degrees

The physics software reads angles in degrees unless told otherwise. The model was written in radians. Every joint limit was therefore read as under 3°, and the arm could barely move.

What it looked like: every algorithm scored zero. Which looks exactly like "machine learning does not work on this problem".

3.2 Defect 2 — the arm dissolved over time

To test robustness, the arm's weight is randomly varied each attempt. The code multiplied the *current* weight rather than resetting to the true weight first.

> Intended: "each attempt, make the arm 85–115 % of its normal weight."
> Implemented: "each attempt, multiply whatever it weighs *now* by 0.85–1.15."

Repeat a thousand times and the arm weighs almost nothing. A drift measure, where 1.0 means no drift, read **0.013**. After the fix: **0.998**.

3.3 Defect 3 — the agent was paid to destroy the arm

This is the most instructive fault in the project.

The agent earns points for its behaviour. Two design choices interacted catastrophically: every point score was zero or negative, and if the simulation broke down, the attempt simply ended with no penalty.

In this kind of learning, an attempt that *ends* earns nothing more — its remaining value is exactly zero. An attempt that runs its full length accumulated about −20 points.

So: breaking the simulation was worth 0. Behaving properly was worth −20. **Destroying the arm was the best possible strategy**, and easier to achieve than success.

The agent found it. A diagnostic over 50 attempts showed **100 % ended in a deliberate breakdown at step 7 of 100**. Not one ran to completion.

Why nobody noticed for weeks: the recorded measurements could not reveal it. The reported distance was still a plausible number, and still slowly improving. Nothing recorded how *long* attempts lasted. An agent that quit after 7 steps of 100 looked identical to one that tried hard and plateaued.

The lesson. The agent optimises exactly what you asked for. When behaviour is strange, suspect the request before the learner. And record enough to tell the difference between "trying and failing" and "not trying".

3.4 Defect 4 — the simulator quietly cleaned up after itself

When the physics breaks down, the software silently resets the arm to its starting position and carries on. By the time our code looked, everything seemed normal — the arm was just mysteriously back at the start.

The measurements therefore recorded the distance from the *starting pose* to the target and filed it as real data.

3.5 Defect 5 — training was doing a quarter of the work

The trainer ran four copies of the simulation at once to gather experience faster. But because of how the learning library counts, each round of four experiences was followed by only *one* round of learning — not four.

Measured directly: **0.233** learning updates per experience, where the standard is about 1.0. **Every experiment had been doing roughly a quarter of its intended training.**

An independent check confirmed it: 233 000 updates at the measured 17 milliseconds each predicts 71 minutes; the runs took 78.8 and 80.6 minutes.

The lesson. Measure what your tools actually do rather than what you believe they do. This defect was invisible in every graph and cost weeks.

3.6 Defect 6 — a standard setting that made precision impossible

After the first five fixes, the arm reached the target region reliably and then stopped improving. The closest it ever got stayed near 0.10 m through *three complete redesigns of the scoring system* and the fourfold training correction.

Finding the cause took two attempts, and the first was wrong.

The wrong answer. A hyperparameter search had changed five settings at once, and one of them — how much deliberate randomness the learner keeps in its actions — moved furthest from its default and had a mechanism that fitted the symptom perfectly. All five of the best trials agreed on roughly the same value. It was written up as the explanation.

Then it was tested properly. Each of the five settings was changed *on its own*, three times each, to see which one actually mattered.

Setting changed alone	Improvement recovered
randomness ("target entropy")	−8 %
learning rate	−3 %
how fast the value estimate updates	−5 %
batch size	−5 %
discount factor	**+106 %**

Four settings did nothing. One did everything — and it was not the one that had been written up.

Why the discount factor. It controls how far ahead the learner looks. The default made it plan over 100 steps: the whole two-second attempt. But the hand actually arrives at the target by step 25. So the learner was being asked to judge each action by what happened over the following two seconds, when only the first half-second was relevant. That is a noisy thing to estimate, and a learner working from noisy judgements cannot fine-tune. Shortening the horizon to match the movement fixed it.

The lesson. Two lessons, and the second is the harder one. First: a setting left at its default is a guess by someone who never saw your problem — here it was four times too long. Second: a hyperparameter search tells you *which combination* works, never *which part* of it was responsible. The explanation that felt obvious was wrong, and only changing one thing at a time revealed it.

4 Step 2: what happened when it finally worked

Table 1: Progress across the four main experiments. "Closest approach" is the nearest the hand ever got; "final error" is where it ended up.

Experiment	Final error	Closest	What changed
Pilot 4	0.242 m	0.178 m	scoring fixed
Pilot 5	0.254 m	0.134 m	scoring refined
Pilot 6	0.204 m	0.132 m	training budget fixed
Final	0.106 m	0.074 m	settings tuned

Notice the third column. Closest approach barely moved across the first three experiments — 0.178, 0.134, 0.132 m — despite huge changes to the scoring system and the training. That refusal to move is what finally pointed at the settings rather than the design.

For scale: a controller *given information the learner never sees* manages about 0.08 m. The tuned policy reaches 0.074 m. It matched a controller that was effectively cheating.

The behaviour changed in kind, not just degree. Before tuning, the hand swept past the target and wandered; the total path travelled was 3.55 m for a reach of 0.80 m. After tuning: 1.64 m, with the hand arriving by step 25 of 100 and *staying*.

5 Step 3: the real test — are the muscle forces right?

Everything above measures *where the hand ended up*. None of it checks whether the *muscles* were doing sensible things — which is the actual research question.

> A student can reach the right answer to a maths problem by a completely wrong method. Checking only the answer would never reveal it.

So at every instant of a movement, we asked the classical method the same question the policy had just answered: *given this exact motion, what is the cheapest set of muscle forces that produces it?* Then we compared, muscle by muscle.

5.1 The result: right pattern, wrong economy

The pattern matches. The correlation between the learned forces and the classical ones is about 0.82. The learner recruits broadly the right muscles in broadly the right proportions.

The economy does not. Producing *identical* joint movements, the learned controller uses about **1.9 times** the muscle effort the classical method says is needed.

And the waste has an address. The shoulder muscles agree closely — one to within 2.5 %. Almost all the excess is in two muscles that straighten the elbow, driven at three to four times the necessary force *while the muscles that bend the elbow are also pulling*. The two fight each other. The elbow ends up roughly right, and both groups are exhausted.

This is called **co-contraction**. Humans do it deliberately when they need a joint to be stiff — carrying something fragile, for instance. Doing it constantly during an ordinary reach is simply wasteful.

5.2 Three independent checks agreed

What makes this trustworthy is that three unrelated methods pointed at the same muscles:

1. the effort comparison above;
2. a standard clinical index of co-contraction, computed from the elbow muscles alone;
3. a decomposition that asks how many independent "muscle groups" the controller really uses — the classical solution needs 4–5, the learned one needs 6–9.

Three methods that share no mathematics, locating the same fault in the same muscles, is much stronger evidence than any one of them alone.

6 Step 4: checking our own claims, and withdrawing two

Once results existed, they were attacked deliberately. Two did not survive.

6.1 A number that flattered itself

The agreement between the learned and classical forces was first reported as a "cosine similarity" of 0.933, which sounds close to perfect.

But muscle forces are never negative — a muscle pulls or does nothing, it never pushes. When all the numbers in two lists are positive, the lists agree substantially *by construction*. Shuffling which muscle each force belonged to — destroying the correspondence entirely — still scored **0.373**.

So roughly the first 0.37 of that 0.933 was floor, not evidence. The honest figure to quote is the correlation (0.82), which does not have this problem. The agreement is real; it was simply being reported in a flattering unit.

6.2 The speed argument, withdrawn entirely

The original motivation was that a neural network answers instantly while the classical method must solve equations step by step. A measurement supported this: 295 µs for the network against 2165 µs for the classical solver, a 7.3× advantage.

Two problems. It timed the wrong calculation — a different piece of the physics, not the muscle-force problem. And both sides were written in unoptimised Python.

Re-measured properly, on the actual muscle-force problem:

Method	**Time**
Neural network	289 µs
Classical, general-purpose solver	2371 µs
Classical, solved directly	24 µs

Done properly, **the classical method is about twelve times faster than the network.** The speed argument was withdrawn from the thesis.

The lesson. The most valuable check is the one that can destroy your own claim. This one did, and the thesis is more trustworthy without it.

7 Step 5: repeating it, because once is not evidence

Every headline number came from a single training run. Training involves randomness, so one run could be lucky. The experiment was repeated with different random starting points.

Table 2: The same configuration, trained five times from different random starts.

Run	**Final error**	**Closest**	**Effort ratio**	**Success**
1 (original)	0.1063	0.0740	1.71	4 %
2	0.1019	0.0742	1.91	20 %
3	0.0988	0.0728	2.12	0 %
4	0.1070	0.0760	1.86	8 %
5	0.1023	0.0676	1.97	6 %
average	**0.1074**	**0.0734**	**1.91**	8 %

Three things came out of this, and only the first was expected.

The accuracy is solid. Final error varies by 3.3 % and closest approach by 4.4 % across runs. That result is a genuine property of the method, not luck.

The original run was the luckiest. On effort it scored 1.71, the best of the four (the others: 1.86, 1.91, 1.97, 2.12). The corrected average is **1.91**. Reporting the single original run had

flattered the result, and the thesis was updated.

The success rate is meaningless. Across five identical configurations it read 0 %, 4 %, 6 %, 8 % and 20 %. A measurement that varies twentyfold on identical settings measures noise. It is reported, with that warning attached, and never used as a headline.

7.1 An unexpected finding

Accuracy barely varied between runs — but the *muscle energy* varied by 77 %. Different runs reached the same place, equally well, using very different patterns of muscle force.

That is the redundancy problem from Section 1, appearing in the results. There really are many different ways to make the same movement, and the learner lands on a different one each time.

8 Step 6: can the waste be removed?

The finding above — right muscles, too much force — raised an obvious question. The scoring system had a term meant to discourage wasted effort. Was it simply set too weakly?

It was. Measured on a trained controller, that term contributed about *one twenty-eighth* of what the accuracy terms contributed. It existed, but it was far too quiet to influence anything.

So the experiment was to turn it up, at two settings, five runs each.

Effort weight	Wasted effort	Agreement with maths	Accuracy
1 (original)	1.91×	0.81	0.1074
5	1.41×	0.92	0.1114
20	**1.30×**	**0.94**	0.1161

The waste fell steadily and agreement with the classical answer rose steadily. Accuracy did suffer, but only slightly: the final distance grew by about 8 % across the whole range, while the closest the hand ever got did not change at all. So the controller became no worse at *reaching* the target — only slightly worse at *staying* there.

For scale: a conventional engineered controller, solving the effort problem explicitly at every instant, scores 1.33 on the same measure. The learned controller at the highest setting reaches 1.30. They are, to the precision available here, equivalent.

There was also a quieter result that mattered more than it looks. At the original setting, five identical runs produced muscle patterns differing by 77 % in total energy while reaching equally well. At the highest setting, that spread almost vanished. The scoring system had never really specified how to share the force, so each run had invented its own answer; saying what we wanted made them agree.

9 What the answer actually is

For *which* muscles to use and in what proportion: yes. The learned controller recovers the structure of the classical solution without ever being shown it.

For *how much* force: yes, once we asked for it. Under the original scoring the controller used about 1.9 times the necessary effort, concentrated in one pair of opposing elbow muscles.

Turning up the term that discourages waste brought that to 1.30× — level with a conventional engineered controller — without costing accuracy.

For speed: no. The classical method, properly implemented, is faster.

9.1 Honest limits

- Everything is simulated. No real arm, no measurements from a human body. The classical method is itself a *theory* of how muscles share load, not ground truth.
- Nine muscles is a heavy simplification; the rotator cuff is absent entirely, which forced one joint to be restrained.
- The comparison is conditional: the classical method is handed the movement the learner produced and asked only how to share the force. It never picks a movement of its own, so nothing here shows the movement itself is natural.
- The task is not solved to the strict standard set at the outset. Under a demanding definition of success, performance is near zero — but that standard is one a cheating controller only meets 7 % of the time, so it was the wrong yardstick.

10 The single most useful thing learned

Four of the six defects produced results that looked entirely reasonable.

> **A measurement that cannot fail cannot detect failure.**

Distance-to-target could not reveal that the agent had stopped trying. It could not reveal that training was doing a quarter of its work. It could not reveal that the muscle forces were 90 % wasteful.

Each of those needed a measurement designed specifically to catch that failure — which means somebody had to imagine the failure first. That, more than any number in this project, is what the work actually taught.